Software Project Management Plan (SPMP)

For

YOLO-BASED REAL-TIME OBJECT DETECTION AND CLASSIFICATION SYSTEM

Submitted By - _____

Date - 08 October 2026

Table of Contents

1. Introduction 3
    1.1. Project Overview 3
    1.2. Project Deliverables 3
    1.3. Evolution of SPMP 4
    1.4. References 4
    1.5. Definitions and Acronyms 4
2. Project Organization 5
    2.1. Process Model 5
    2.2. Organizational Structure 5
    2.3. Organizational Boundaries 6
    2.4. Project Responsibilities 6
3. Managerial Process 7
    3.1. Management Objectives & Priorities 7
    3.2. Assumptions, Dependencies & Constraints 7
    3.3. Risk Management 8
    3.4. Monitoring and Controlling Mechanisms 8
    3.5. Staffing Plan 8
4. Technical Process 9
    4.1. Methods, Tools and Techniques 9
    4.2. Software Documentation 9
    4.3. Project Support Functions 9
5. Work Packages, Schedule, and Budget 10
    5.1. Work Packages 10
    5.2. Dependencies 10
    5.3. Resource Requirements 10
    5.4. Budget and Resource Allocation 11
    5.5. Schedule 11
6. Additional Components 12
    6.1. Index 12

1. Introduction

This document refers to the planning and management of the software project titled YOLO-Based Real-Time Object Detection and Classification System. The section provides a basic overview of the proposed system, its deliverables, and the controls that will be used to manage development. The project is planned as a practical real-time computer vision application in which an AI detection model is integrated into a complete software product rather than being used only as a standalone demonstration.

1.1 Project Overview

The objective of this project is to design and develop a real-time object detection and classification system capable of receiving images, video files, and live camera streams and identifying multiple objects present in the input. The planned implementation uses the current released Ultralytics YOLO26 model family. YOLO26 was released in January 2026 and provides detection, classification, tracking, export, validation and other capabilities through the Ultralytics framework. It introduces a dual-head design, optional end-to-end NMS-free inference, a lighter detection head, and an updated training recipe using MuSGD, Progressive Loss and STAL. The official documentation describes YOLO26 as the newest released Ultralytics YOLO family; YOLO27 is currently listed as an upcoming family rather than a released model. [1][2]

The system will process each input frame using the selected YOLO26 detection model and return the object class, confidence value and bounding-box coordinates. The user interface will visualize the results in real time. Where tracking is enabled, detected objects will retain persistent identifiers across video frames, allowing object counting and event-oriented analytics. Ultralytics currently provides multiple built-in trackers, including TrackTrack, BoT-SORT, ByteTrack, OC-SORT, Deep OC-SORT and FastTracker. [3]

The project is intended to demonstrate software engineering practices including requirements analysis, incremental development, modular architecture, database management, REST-based services, automated testing, version control, performance measurement, model evaluation and deployment. The AI model is therefore treated as one technical component inside a larger software system.

1.2 Project Deliverables

- The complete source code for the frontend, backend, detection service, tracking/counting modules and database integration.
- A configured YOLO26 model and, where applicable, the fine-tuned project-specific model weights and training configuration.
- Dataset documentation covering data source, class definitions, annotation procedure, training/validation/test split and preprocessing.
- Software design documents including system architecture, use cases, database design, API specification and project plan.
- A functional web-based prototype supporting image, video and real-time camera inputs.
- Test documentation, test cases, defect records, model evaluation results and performance benchmark results.
- User documentation describing system operation, configuration, limitations and basic troubleshooting.
- Final integrated product and deployment package for demonstration or hosted use.

1.3 Evolution of SPMP

The requirements for the object detection system shall be identified and reviewed before the implementation of each increment. Changes involving the model family, supported input types, database schema, tracking logic, security requirements or deployment environment shall be recorded in the SPMP or associated change log before implementation. Since AI-system performance can depend on the dataset, hardware and inference settings, changes to model version, model size, confidence threshold, input resolution or export format shall also be treated as controlled technical changes.

The baseline for this plan is YOLO26 and the Ultralytics software stack available at the date of this document. Official documentation states that YOLO26 supports five detection scales (n, s, m, l and x) and can be evaluated, trained, exported and used for tracking. The project team shall record the exact model checkpoint, software package version, hardware configuration and dataset version used for each formal experiment so that the reported results remain reproducible. [1][2][3]

1.4 References

- [1] Ultralytics, “Ultralytics YOLO26,” official documentation, released January 2026 and accessed 08 October 2026. https://docs.ultralytics.com/models/yolo26
- [2] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon and M. E. Kalfaoglu, “Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models,” arXiv:2606.03748, 2026. https://arxiv.org/abs/2606.03748
- [3] Ultralytics, “Multi-Object Tracking with Ultralytics YOLO,” official documentation, accessed 08 October 2026. https://docs.ultralytics.com/modes/track
- [4] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” Proceedings of CVPR, 2016. https://arxiv.org/abs/1506.02640
- [5] Ultralytics, “YOLO26 Modes,” official documentation, accessed 08 October 2026. https://docs.ultralytics.com/modes
- [6] Institute of Electrical and Electronics Engineers, IEEE 1058, Software Project Management Plans, reference framework for project management planning.

1.5 Definitions and Acronyms

Term	Definition
YOLO	You Only Look Once; a family of real-time deep-learning models for computer vision tasks.
YOLO26	Current released Ultralytics YOLO model family used as the baseline AI component in this project.
SPMP	Software Project Management Plan.
CV	Computer Vision.
AI	Artificial Intelligence.
NMS	Non-Maximum Suppression, a post-processing method used to remove redundant overlapping detections.
IoU	Intersection over Union; overlap measure between predicted and ground-truth bounding boxes.
mAP	Mean Average Precision; a standard metric family used to evaluate object-detection accuracy.
FPS	Frames Per Second; number of video frames processed per second.
REST	Representational State Transfer; style used for HTTP-based application programming interfaces.
API	Application Programming Interface.
QA	Quality Assurance.
ReID	Re-Identification; appearance-based matching used by some object trackers.

2 Project Organization

This section describes the process model to be used for the project and the proposed role structure. Because the project combines conventional software development with machine-learning experimentation, the organization must support both iterative engineering and controlled model evaluation.

2.1 Process Model

The process model to be followed in this project is the incremental model. This model is selected because the system requirements can be divided into relatively independent modules. Requirements gathering, analysis and design, implementation, testing and deployment will be performed in increments. Each increment will deliver a usable extension of the previous release while keeping a stable baseline for the already completed functionality.

The planned increments are: Increment 1 - core image detection and result visualization; Increment 2 - live webcam and video processing; Increment 3 - tracking and counting; Increment 4 - user accounts, history and database persistence; Increment 5 - analytics, performance benchmarking, optimization and deployment. Model experimentation is interleaved with development so that the selected YOLO26 variant can be changed without redesigning the complete application.

2.2 Organizational Structure

The proposed organization is an academic software project team. A faculty/project supervisor provides oversight and approves major milestones. A project manager/team lead coordinates requirements, schedule, integration and review. The AI/computer-vision developer handles model selection, dataset preparation, training and evaluation. The full-stack/database developer implements the web application, REST services and persistence layer. A QA/documentation role prepares test cases, defect records, user documentation and release evidence. In a smaller student team, multiple roles may be performed by the same person while maintaining the same responsibilities.

Role	Primary Responsibility
Project Supervisor / Sponsor	Milestone approval, academic oversight, major requirement decisions and escalation.
Project Manager / Team Lead	Planning, work assignment, progress tracking, reviews, integration and stakeholder communication.
AI / Computer-Vision Developer	YOLO26 selection, dataset preparation, model training/fine-tuning, evaluation and inference optimization.
Full-Stack / Database Developer	Frontend, Flask/API services, database schema, authentication and application integration.
QA / Documentation	Test planning, automation, defect tracking, verification evidence, user guide and final documentation.

Git and GitHub will be used to maintain the source-code baseline and provide traceable change history. Issue tracking shall be used for requirements, defects and implementation tasks. Major releases shall be tagged so that model experiments and software releases can be reproduced.

2.3 Organizational Boundaries

All project requirements are coordinated through the project manager/team lead and reviewed with the project supervisor. Technical decisions affecting the AI model, database, application architecture or deployment environment must be recorded in the project decision log. Any change likely to affect a milestone, scope, budget, privacy posture or evaluation criteria shall be escalated before implementation.

The AI workstream is responsible for the correctness and reproducibility of model experiments, but the AI workstream does not independently define end-user requirements. The software workstream owns application behavior, access control, storage and integration. QA verifies that both the application and model-related acceptance criteria are met. The final project decision remains with the supervisor/team lead according to the academic context.

2.4 Project Responsibilities

Role	Responsibilities
Project Supervisor / Sponsor	Review scope and milestones; approve major changes; assess whether project outcomes meet academic objectives.
Project Manager / Team Lead	Create schedule; assign tasks; maintain risk register; chair progress meetings; approve module readiness; coordinate final integration.
AI / CV Developer	Prepare datasets; configure YOLO26 experiments; document hyperparameters; train/fine-tune models; validate precision/recall/mAP; monitor inference performance.
Full-Stack / Database Developer	Develop UI; implement REST endpoints; integrate inference service; manage authentication; design and maintain MySQL schema; implement analytics APIs.
QA / Documentation	Prepare unit and integration test cases; perform system, usability and performance tests; record defects; maintain user and technical documentation.

The team shall use a shared repository with a protected main branch. Feature work should be isolated through branches or pull requests. Code review is expected before integrating major changes. Model checkpoints, dataset versions and experiment metadata should be stored separately from application source code when their size makes direct repository storage impractical.

The project boundary includes software necessary to ingest data, run inference, present results, track objects, persist session-level records and report metrics. The project does not include the development of a new object-detection algorithm from first principles; the research component is centered on the controlled selection, adaptation, evaluation and integration of the YOLO26 model family.

3 Managerial Process

This section of the SPMP specifies the management process for the project. Management activities are intended to balance software scope, AI accuracy, real-time performance, schedule, maintainability and testing quality.

3.1 Management Objectives & Priorities

The overall objective of management is to deliver a stable and demonstrable real-time object detection application within the agreed project duration and resources. The highest priority is a reliable core detection workflow. Secondary priorities include tracking, counting, analytics, user management, performance optimization and deployment. If schedule pressure occurs, optional enhancements are reduced before the core detection, testing and documentation activities are reduced.

Management priorities are therefore ordered as follows: correctness and safety of software behavior; completion of the core AI inference workflow; reproducible model evaluation; real-time responsiveness; database and analytics features; advanced deployment optimization. No feature shall be accepted solely because it works in one demonstration. It must satisfy the corresponding requirement and have evidence from testing or evaluation.

Daily or near-daily team coordination will be used during active development. A weekly formal review will examine completed work, upcoming tasks, risks, defects, model metrics and schedule variance. A milestone review will be required before each major release is considered complete.

3.2 Assumptions, Dependencies & Constraints

- The system will have access to a compatible Python runtime, a webcam for live demonstrations and sufficient CPU/GPU resources for the selected YOLO26 model.
- The primary implementation will use a released YOLO26 checkpoint; custom fine-tuning is dependent on the availability of an appropriately licensed and sufficiently annotated dataset.
- The application requires a stable software environment with the required Python, PyTorch, OpenCV, Flask, database and frontend dependencies.
- The direct software-license cost is expected to be low for an academic prototype because the planned stack primarily uses open-source software; however, current licenses must be checked before any public/commercial deployment.
- Real-time performance will depend on hardware, model size, input resolution, camera frame rate and whether the model is exported to an optimized runtime.
- The system should be modular so that the YOLO model, tracker or database implementation can be replaced without rewriting the entire application.
- The project shall protect stored credentials and minimize unnecessary retention of camera data.

The current Ultralytics documentation lists AGPL-3.0 and Enterprise licensing options and explicitly distinguishes licensing for different usage contexts; the academic project shall therefore retain a license notice and verify obligations for the final deployment scenario. [6]

3.3 Risk Management

Risk factors and the tracking mechanisms for the project include the following. Each risk will be recorded in a risk register with probability, impact, owner, mitigation and contingency actions.

Risk	Prob.	Impact	Mitigation	Contingency
Low real-time FPS	Medium	High	Use smaller YOLO26 model; reduce input size; measure latency; consider optimized export.	Fall back to YOLO26n or process selected frames.
Poor accuracy on custom classes	Medium	High	Improve annotations; add representative data; balance classes; fine-tune and validate.	Use pretrained classes and narrow the project scope.
Small/occluded objects missed	Medium	High	Use appropriate training data and augmentation; evaluate small-object cases separately.	Document limitation and adjust camera/input constraints.
Webcam/video incompatibility	Medium	Medium	Test multiple codecs and devices; use OpenCV validation.	Provide image upload mode as a reliable fallback.
Model/package incompatibility	Low	High	Pin dependencies; record exact versions; test after upgrades.	Freeze the last verified environment.
Database or API failure	Low	High	Input validation, transactions, error handling and backups.	Queue or temporarily save session summaries locally.
Privacy/security issue	Low	Very High	Hash passwords, limit retention, authenticate APIs and avoid unnecessary raw video storage.	Disable affected module, preserve logs and correct before release.
Schedule overrun	Medium	High	Incremental releases; prioritize core features; weekly progress review.	Defer optional analytics or deployment enhancements.

3.4 Monitoring and Controlling Mechanisms

The team lead will assign work and monitor progress against the baseline schedule. Task status shall be maintained in the issue tracker. Major problems affecting the AI model, application stability, schedule or scope shall be reported immediately. The project manager will periodically review the team status, defect count, completed requirements, model evaluation metrics and outstanding risks with the project supervisor.

The following indicators will be monitored: percentage of functional requirements completed; number and severity of open defects; test pass rate; model precision, recall and mAP; average inference latency; average FPS; number of failed inference sessions; and schedule variance. A release is not considered complete if critical defects remain open or if required evaluation evidence has not been collected.

3.5 Staffing Plan

The project is planned for 24 weeks. The project supervisor is needed at milestones and scheduled reviews. The project manager/team lead is required throughout the project because planning, integration, reviews and risk management continue across all increments. The AI and full-stack developers are most heavily utilized from the dataset/model and implementation stages through integration. QA/documentation effort increases during system testing and final release preparation.

4 Technical Process

This section includes the technical methods, tools and techniques to be used for development of the YOLO-based system. The implementation will separate application logic from AI inference so that the model can be upgraded or replaced without redesigning the complete product.

4.1 Methods, Tools and Techniques

The incremental software process will be supported by UML-based design, Git version control and issue tracking. The primary AI framework will be Ultralytics YOLO26 with Python. The current YOLO26 family contains five detection scales - n, s, m, l and x - and supports training, validation, prediction, export, tracking and benchmarking. The default detection path uses the one-to-many head with NMS, while the one-to-one head can be selected with nms=False for NMS-free inference. [1][5]

For this project, YOLO26n will be used as the initial real-time baseline and YOLO26s may be evaluated as an accuracy-oriented comparison when hardware permits. The final selection will be based on measured accuracy and latency rather than model size alone. Ultralytics reports COCO benchmark ranges from 40.9 to 57.5 mAP50-95 across YOLO26n through YOLO26x, with published T4 TensorRT latency from 1.7 to 11.8 ms under the documented benchmark conditions; these figures are reference benchmarks and shall not be presented as project results. [1]

OpenCV will handle camera capture, frame decoding, resizing and result visualization. Flask will provide backend routes and REST APIs. HTML, CSS and JavaScript will form the web interface. MySQL will store user accounts, detection sessions, summary statistics and event data. Chart.js or an equivalent browser charting library will be used for analytics. pytest will support automated unit tests, while API and system tests will verify end-to-end behavior.

Object tracking will be implemented through the Ultralytics track mode. The current framework supports TrackTrack as the default tracker and also provides BoT-SORT, ByteTrack, OC-SORT, Deep OC-SORT and FastTracker. ByteTrack is suitable as a simple low-overhead starting point, while BoT-SORT can be considered for moving-camera scenarios; the final tracker shall be selected through project testing. [3]

4.2 Software Documentation

Software documentation will be created throughout development rather than only at the end. Each module shall have a short description of its inputs, outputs, dependencies and error behavior. The documentation set will contain the SPMP, requirements specification, UML diagrams, database schema, API documentation, installation instructions, dataset/model documentation, test cases, benchmark methodology, user guide and release notes. Changes to documentation will be reviewed during milestone reviews so that the written plan remains synchronized with implementation.

4.3 Project Support Functions

Support functions include source-code version control, dependency management, environment reproducibility, dataset versioning, model checkpoint management, issue tracking, test execution, database backup and deployment support. The project repository will contain configuration templates but shall not contain secrets such as production passwords or API keys. A release checklist will verify installation, inference, camera access, database connectivity, tests and documentation before the final demonstration.

5 Work Packages, Schedule, and Budget

This section defines the planned work packages, dependencies and resource requirements for the project. The schedule uses incremental delivery so that an executable system is available before the final stage.

5.1 Work Packages

Release	Scope
First Release - Core Detection	Requirements, architecture, environment setup, pretrained YOLO26 inference, image upload, bounding boxes, class labels and confidence display.
Second Release - Real-Time Input	Webcam and video processing, frame management, FPS display, confidence threshold control and error handling.
Third Release - Tracking and Counting	Persistent object IDs, tracker selection, line-crossing logic, object counts and event generation.
Fourth Release - Platform Features	Authentication, database persistence, detection history, analytics dashboard and configurable settings.
Fifth Release - Evaluation and Deployment	Custom dataset/fine-tuning where applicable, accuracy evaluation, performance benchmarking, optimization, export, documentation and final deployment.

The first release establishes the technical baseline. The second release proves real-time operation. The third release demonstrates the transformation from frame-by-frame detection into video analytics. The fourth release demonstrates complete application-level functionality. The final release concentrates on evidence, reproducibility, optimization and deployment.

5.2 Dependencies

- The requirements baseline must be approved before detailed design is finalized.
- The selected model checkpoint and compatible Python environment must be available before inference integration begins.
- Training or fine-tuning cannot begin until the dataset has been checked for class definitions, annotations, licensing and quality.
- Tracking depends on a stable detection output pipeline and valid frame processing.
- Analytics depend on the database schema and session/event logging mechanisms.
- Formal performance benchmarking depends on a frozen software/model configuration and documented hardware.
- Deployment depends on successful system testing, configuration management and license review.

5.3 Resource Requirements

Resource	Requirement
Development computer	Recommended minimum 16 GB RAM; modern multi-core CPU; webcam; GPU optional but useful for training and high-throughput inference.
Software	Python, PyTorch, Ultralytics, OpenCV, Flask, MySQL, browser runtime, Git, testing libraries and supporting packages.
Dataset storage	Local or cloud storage sufficient for original images, annotations, derived splits and model artifacts.
Version control	Git repository with protected main branch and tagged releases.
Optional acceleration	CUDA-capable NVIDIA GPU, compatible accelerator, or exported runtime such as ONNX/TensorRT depending deployment target.
Documentation tools	Word processor/Markdown, diagram tool and spreadsheet/charting utilities for evidence and schedules.

5.4 Budget and Resource Allocation

The following is an estimated academic-project direct-cost plan. It excludes student salaries and assumes that development computers are already available. The planned software stack is primarily open source, but current license terms must be checked for the intended deployment. Optional cloud GPU and hosting costs are included only as contingency resources.

Phase	People / Role	No. of People	Hours/Days	Budget
Requirements Gathering	Project Manager / Supervisor	1-2 people	12 hours	$0
Design and Analysis	Team Lead + Developers	3 people	30 hours	$0
Implementation	AI + Full-Stack + Support	3 people	180 hours	$0
Testing and Evaluation	QA + Developers	2-3 people	50 hours	$0
Deployment and Documentation	Team	3-5 people	40 hours	$0
Optional Cloud GPU / Hosting / Storage	Deployment / Infrastructure	As required	Usage-based	$370 contingency
Total Estimated Direct Cost	Project	5 role assignments	312 hours + optional usage	$370 maximum planned direct cost

The direct-cost estimate can be replaced with the institution-specific costing format if the course requires labor rates. Any paid cloud service will be approved before use and recorded with the actual amount in the project cost log.

5.5 Schedule

The proposed project duration is 24 weeks. Each release is treated as a milestone. Work packages overlap where dependencies permit, while formal acceptance checks are retained at the end of each increment. The Gantt chart below summarizes the baseline schedule; dates may be shifted while preserving the dependency order and milestone logic.

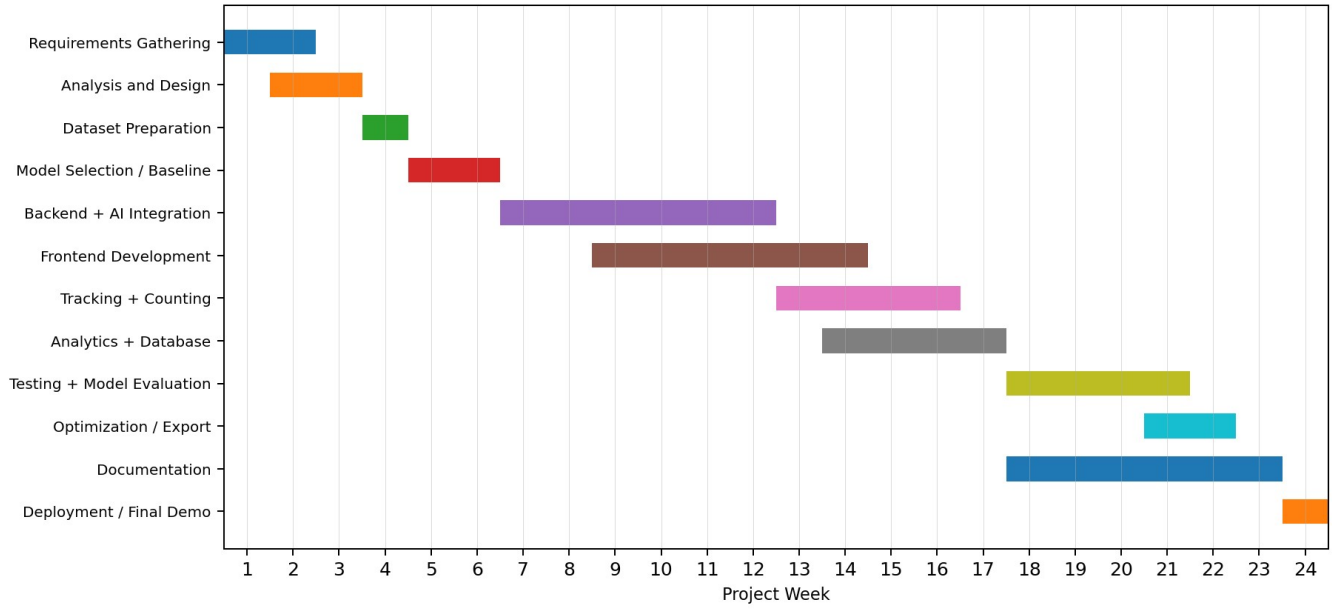


Figure 1: Timeline for project

Model evaluation is treated as a formal project activity rather than an optional final step. Before the final release, the team shall freeze the model/software configuration used for reporting and record the test dataset, hardware, image size, confidence settings and runtime used to produce the reported measurements.

6 Additional Components

This section includes additional components required to make the product usable and maintainable after implementation.

6.1 Index

Security - User accounts will use authenticated access. Passwords will be stored as salted hashes rather than plain text. API routes will validate authorization before returning protected history or analytics. Uploaded files will be checked for supported type and size. Raw camera/video retention will be minimized and configured according to the project requirements. Logs will avoid storing credentials or unnecessary personal information.

Training - A walkthrough of the system will be provided to users demonstrating login, image upload, webcam operation, confidence threshold settings, tracking/counting controls, history and analytics. Technical training for the project team will cover the repository workflow, environment setup, model loading, dataset structure, model evaluation and troubleshooting. Training material will be included in the user guide.

Product Maintenance - Periodic checking will be provided for the project software, model checkpoint, Python dependencies, browser compatibility and database. Any update to YOLO, PyTorch, OpenCV or the deployment runtime shall be tested before being accepted because software upgrades can change model behavior, performance or export compatibility. Model changes will be treated as controlled releases and shall retain the previous validated checkpoint for rollback.

Model and Data Governance - Every formal experiment shall record model name, model scale, checkpoint, dataset version, training configuration, image size, confidence/IoU settings, hardware and evaluation date. Dataset licenses and usage restrictions shall be recorded with the dataset metadata. Results shall distinguish between published benchmark figures and measurements obtained by this project.

Quality Assurance - The final product shall not be considered complete until functional tests, integration tests, performance checks and AI evaluation evidence are complete. Critical defects must be resolved or formally accepted by the supervisor before final release. Test evidence shall include both normal cases and failure cases such as invalid files, unavailable cameras, empty detections and low-confidence predictions.

Deployment and Support - The released application will be packaged with installation instructions and configuration guidance. Where an optimized runtime is used, the exact exported model format and runtime version will be documented. Ultralytics currently documents export options such as ONNX, TensorRT, OpenVINO and CoreML among other targets; the chosen deployment format will be selected after benchmarking the target hardware. [1][5]

Reference to Current Model Status - As of 08 October 2026, YOLO26 is the latest released Ultralytics YOLO family. The official model documentation describes YOLO27 as an upcoming family and states that YOLO26 provides the current released baseline. Therefore, the project baseline is explicitly fixed to YOLO26 unless the supervisor approves a later model update. [1][2]

Conclusion - The planned system combines real-time computer vision with a complete software engineering lifecycle. The project is designed to produce a demonstrable application, measurable AI results and maintainable software artifacts. The incremental plan provides a reliable fallback path: even if advanced features are delayed, the core detection and classification system remains a complete working release.